

THIS AND SUPER KEYWORD IN JAVA

Introduction

Java provides two important reference keywords:

- **this keyword** → Refers to the **current object (current class instance)**
- **super keyword** → Refers to the **parent class object**

Both are reference variables used in **Object-Oriented Programming (OOP)**, especially in **inheritance and constructors**.

1. this Keyword in Java

Definition

The **this keyword** refers to the **current object of a class**.

It is used to access:

- Instance variables
- Methods
- Constructors of the same class

Syntax

```
this.variableName;  
this.methodName();  
this();
```

Uses of this Keyword

1. Access Instance Variables

Used when local variable and instance variable have the same name.

```
class Student {  
    int id;  
  
    void setId(int id) {  
        this.id = id;  
    }  
}
```

Here:

- **this.id** → instance variable
- **id** → parameter

Without this, Java cannot distinguish variables.

2. Call Current Class Method

```
class Demo {  
    void show() {  
        this.display();  
    }  
  
    void display() {  
        System.out.println("Hello");  
    }  
}
```

3. Call Current Class Constructor

```
class Demo {  
    Demo() {  
        this(10);  
    }  
  
    Demo(int x) {  
        System.out.println(x);  
    }  
}
```

4. Pass Current Object

```
void method1(Demo obj) { }
```

```
void method2() {  
    method1(this);  
}
```

5. Return Current Object

```
Demo getObject() {  
    return this;  
}
```

2. super Keyword in Java

Definition

The **super keyword** refers to the **immediate parent class object**.

It is used only in **inheritance**.

Syntax

```
super.variableName;  
super.methodName();  
super();
```

Uses of super Keyword

1. Access Parent Class Variable

```
class Parent {  
    int x = 10;  
}
```

```
class Child extends Parent {  
    int x = 20;  
  
    void show() {  
        System.out.println(super.x);  
    }  
}
```

Output:

10

super.x accesses parent variable.

2. Call Parent Class Method

```
class Parent {
    void show() {
        System.out.println("Parent Method");
    }
}

class Child extends Parent {
    void show() {
        super.show();
        System.out.println("Child Method");
    }
}
```

3. Call Parent Class Constructor

```
class Parent {
    Parent() {
        System.out.println("Parent Constructor");
    }
}

class Child extends Parent {
    Child() {
        super();
        System.out.println("Child Constructor");
    }
}
```

If not written, Java automatically calls super().

ABSTRACT CLASS IN JAVA

1. Introduction

An **Abstract Class** in Java is a class that **cannot be instantiated (cannot create objects)** and is meant to be inherited by other classes.

It is declared using the **abstract keyword**.

Abstract classes are used to implement the concept of **abstraction in Object-Oriented Programming (OOP)**.

Example Syntax

```
abstract class Shape {
    abstract void area();
}
```

2. Abstract Method

An **abstract method** is a method **without implementation (without body)**.

Syntax

```
abstract void display();
```

Characteristics:

- No method body
- Ends with semicolon ;
- Must be implemented in child class

If a class contains an abstract method, the class must be declared abstract.

3. Example of Abstract Class

```
abstract class Animal {
    abstract void sound();

    void eat() {
        System.out.println("Animal eats food");
    }
}

class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Main {
    public static void main(String args[]) {
        Dog d = new Dog();
        d.sound();
        d.eat();
    }
}
```

Explanation:

- Animal → Abstract class
 - sound() → Abstract method
 - eat() → Normal method
 - Dog → Child class
-

4. Features of Abstract Class

Important features:

- Declared using **abstract keyword**
 - Cannot create objects
 - Can contain abstract and normal methods
 - Used as a blueprint for child classes
 - Supports inheritance
-

5. Rules of Abstract Class

Rule 1

Abstract class cannot be instantiated.

Animal a = new Animal(); // Error

Rule 2

Child class must implement abstract methods.

```
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

Otherwise child class must also be abstract.

Rule 3

Abstract class can contain:

- Variables
- Constructors
- Static methods
- Final methods

Example:

```
abstract class Demo {  
    Demo() {  
        System.out.println("Constructor");  
    }  
}
```

JAVA INTERFACES

1. Definition

An **Interface** in Java is a **collection of abstract methods** used to achieve **abstraction**.

It is a **reference type similar to a class**, but methods usually do not have implementation.

An interface acts like a **contract** that specifies what a class should do.

2. Syntax of Interface

Interfaces are declared using the keyword:

interface

Example

```
public interface Shape {
```

```
    double PI = 3.14;
```

```
    void area();
```

```
    void perimeter();
```

```
}
```

An interface may contain:

- Abstract methods
 - Constants
 - Default methods
 - Static methods
-

3. Implementing an Interface

A class uses the keyword **implements** to use an interface.

Example

```
interface Animal {  
    void sound();  
}  
  
class Dog implements Animal {  
  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

The class must implement **all methods** of the interface.

4. Important Features of Interface

Features:

- Used to achieve abstraction
 - Cannot create objects
 - Contains method declarations
 - Supports multiple inheritance
 - Similar to class
-

5. Default Properties of Interface Members

In an interface:

- All methods are **public and abstract** by default.
- All variables are **public static final** by default.

Example:

```
interface Demo {  
  
    int x = 10;    // public static final  
    void show();  // public abstract  
  
}
```

6. Interface with Default Method

From Java 8, interfaces can have **default methods** with implementation.

Example

```
interface Vehicle {  
  
    default void show() {  
        System.out.println("Vehicle");  
    }  
}
```

The implementing class **does not need to override default methods**.

7. Multiple Interface Implementation

A class can implement **more than one interface**.

Example

```
interface A {  
    void show();  
}
```

```
interface B {  
    void display();  
}
```

```
class Test implements A,B {  
  
    public void show() {  
        System.out.println("Show");  
    }  
  
    public void display() {  
        System.out.println("Display");  
    }  
}
```

Interface vs Abstract Class

Feature	Interface	Abstract Class
Methods	Mostly abstract	Abstract + Normal
Variables	Constants only	Normal variables allowed
Constructor	Not allowed	Allowed
Object Creation	Not allowed	Not allowed
Inheritance	Multiple	Single